

Самостійна робота №14: Моніторинг та логування застосунків

Тема: Моніторинг та логування застосунків

Мета: Вивчити підходи до моніторингу та логування Node.js-застосунків, ознайомитися з PM2, бібліотеками Winston/Pino, health checks та базовим моніторингом.

Кількість годин: 2

Теоретичний матеріал

PM2 - менеджер процесів

PM2 - менеджер процесів для Node.js-застосунків у продакшні. Основні можливості: автоматичний перезапуск при падінні, cluster mode (використання всіх ядер CPU), zero-downtime reload, моніторинг CPU та пам'яті, логування.

Основні команди: `pm2 start app.js`, `pm2 list` (список процесів), `pm2 logs` (перегляд логів), `pm2 monit` (моніторинг у реальному часі), `pm2 restart`, `pm2 stop`, `pm2 delete`. Файл конфігурації `ecosystem.config.js` дозволяє описати налаштування для кількох застосунків.

Cluster mode: `pm2 start app.js -i max` - запускає по одному процесу на кожне ядро CPU, PM2 автоматично розподіляє навантаження між ними.

Логування - Winston

Winston - найпопулярніша бібліотека логування для Node.js. Концепції: **рівні логів** (error, warn, info, http, verbose, debug, silly), **transports** (куди писати логи: console, file, HTTP), **формати** (JSON, colorize, timestamp, printf).

```
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error'
  }),
    new winston.transports.File({ filename: 'combined.log' }),
  ],
});
```

У Nest.js Winston інтегрується через `nest-winston` як заміна вбудованого Logger.

Логування - Pino

Pino - швидка бібліотека логування (у 5-10 разів швидша за Winston). Генерує JSON-логи, які потім обробляються через `pino-pretty` (для розробки) або транспортуються у системи агрегації (для продакшну). Nest.js інтеграція через `nestjs-pino`.

Health Checks

Health check - ендпоінт, що повідомляє про стан застосунку. Використовується балансувальниками навантаження, Docker, Kubernetes для визначення, чи працює застосунок. У Nest.js - модуль `@nestjs/terminus`:

- **Liveness check** - застосунок працює (не завис, не впав).
- **Readiness check** - застосунок готовий приймати запити (підключений до БД, кеш доступний).

Перевірки: підключення до БД (`TypeOrmHealthIndicator`), дисковий простір (`DiskHealthIndicator`), пам'ять (`MemoryHealthIndicator`), зовнішні сервіси (`HttpHealthIndicator`).

Базовий моніторинг

Метрики для відстеження: час відповіді (response time), кількість запитів (requests per second), використання CPU та пам'яті, кількість помилок (error rate). Інструменти: PM2 monit (базовий), Prometheus + Grafana (розширений), Application Performance Monitoring (APM) - Datadog, New Relic.

Контрольні питання

1. Які основні функції PM2 і чому він потрібен у продакшні?
2. Що таке cluster mode у PM2 і яку перевагу він дає?
3. Які рівні логування існують у Winston і коли використовувати кожен?
4. Чим Pino відрізняється від Winston? У яких сценаріях обрати кожен бібліотеку?
5. Що таке health check і яка різниця між liveness та readiness check?
6. Як налаштувати health check у Nest.js за допомогою `@nestjs/terminus`?
7. Які ключові метрики слід відстежувати у продакшн-застосунку?

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.

Рекомендовані джерела

- PM2 - офіційна документація - повна документація PM2
- Winston - GitHub - документація Winston
- Pino - документація - офіційна документація Pino
- Nest.js - Health Checks - гайд по health checks у Nest.js
- Nest.js - Logger - вбудований логер Nest.js
- 12 Factor App - Logs - принципи логування у хмарних застосунках